



TEMPLATE METHOD PATTERN

1 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành, sinh viên có thể:

- ✓ Trình bày định nghĩa Template Method Pattern
- ✓ Giải thích vai trò của abstract class và hook method
- ✓ Phân biệt Template Method với Strategy
- ✓ Nhận diện khi nào quy trình xử lý có cấu trúc cố định nhưng bước chi tiết thay đổi

2 VẤN ĐỀ (PROBLEM) – ORDER WORKFLOW

Giả sử hệ thống có nhiều loại đơn hàng: `OnlineOrder`, `InStoreOrder`, `InternationalOrder`. Mỗi loại có quy trình: `Validate`, `Calculate Total`, `Process Payment`, `Ship`, `Notify`

✗ Thiết kế cũ – Duplicate code

```
public class OnlineOrder {  
    public void Process() {  
        Validate(); CalculateTotal(); ProcessPayment(); Ship(); SendEmail();  
    }  
}  
  
public class InStoreOrder {  
    public void Process() {  
        Validate(); CalculateTotal(); ProcessPayment(); PrintReceipt();  
    }  
}
```

🚧 Vấn đề: Lặp lại cấu trúc `Process()`, Khó kiểm soát thứ tự, Vi phạm DRY

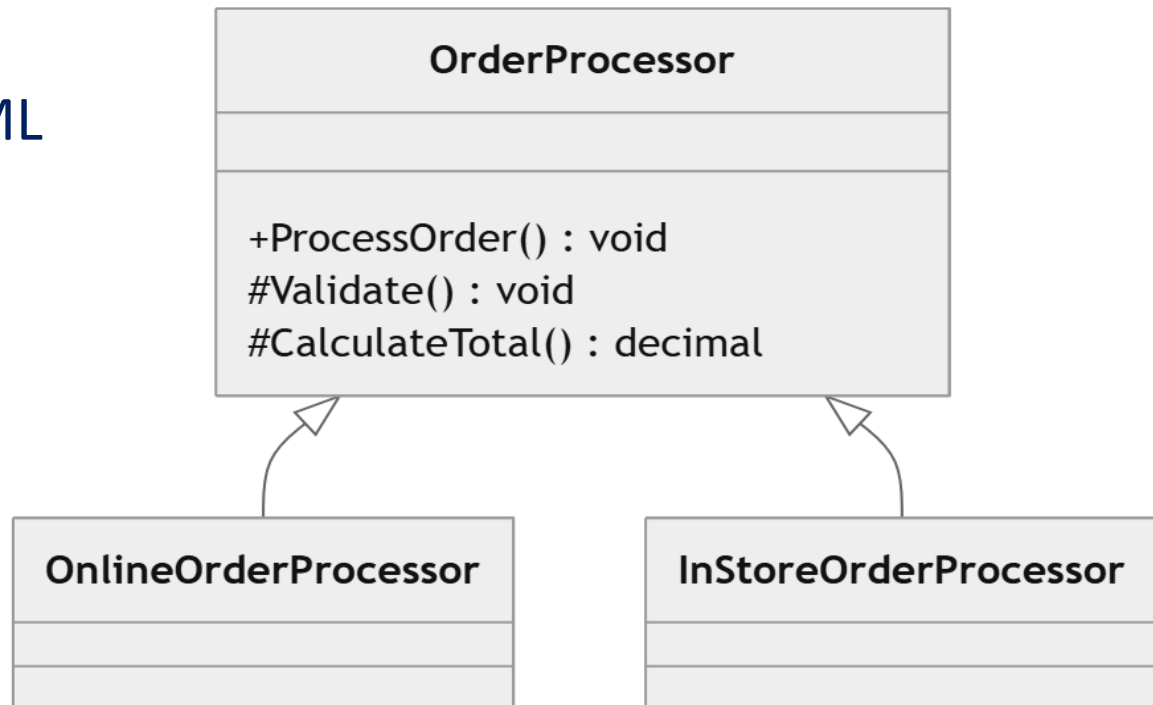
3 GIẢI PHÁP & 4 CẤU TRÚC UML

Định nghĩa

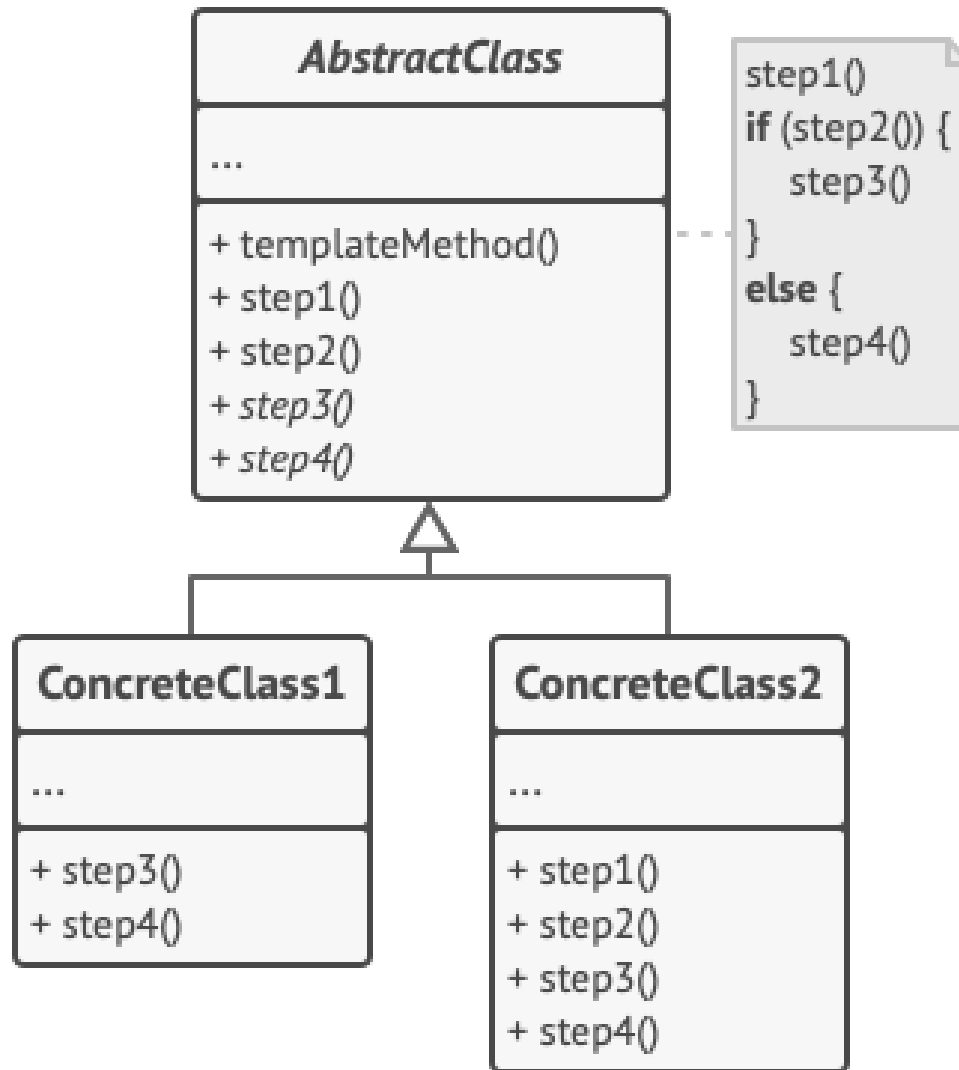
Define the skeleton of an algorithm in an operation, deferring some steps to subclasses.

 Lớp cha định nghĩa khung quy trình, lớp con override các bước cụ thể.

4 CẤU TRÚC UML



Structure of Template Method Pattern



5 TRIỂN KHAI TEMPLATE METHOD (Step 1)

Step 1 – Abstract Template

```
public abstract class OrderProcessor
{
    public void ProcessOrder() {
        Validate(); CalculateTotal(); ProcessPayment(); Ship();
Notify();
    }
    protected abstract void Validate();
    protected abstract void CalculateTotal();
    protected abstract void ProcessPayment();
    protected abstract void Ship();
    protected virtual void Notify() {
        Console.WriteLine("Default notification sent.");
    }
}
```

5 TRIỂN KHAI TEMPLATE METHOD (Step 2 & 3)

Step 2 – OnlineOrderProcessor

```
public class OnlineOrderProcessor : OrderProcessor {  
    protected override void Validate() { /* ... */ }  
    protected override void Ship() { Console.WriteLine("Shipping via  
courier..."); }  
}
```

Step 3 – InStoreOrderProcessor (Hook Method override)

```
public class InStoreOrderProcessor : OrderProcessor {  
    protected override void Ship() { Console.WriteLine("No  
shipping required."); }  
    protected override void Notify() {  
        Console.WriteLine("Printing receipt..."); }  
}
```

6 & **7** SO SÁNH VÀ PHÂN BIỆT

BEFORE	AFTER
Duplicate Process()	Skeleton centralized
Khó kiểm soát flow	Flow cố định
Vi phạm DRY	Tuân thủ DRY
Sửa nhiều nơi	Sửa 1 nơi

Template Method	Strategy
Dùng inheritance	Dùng composition
Khung cố định	Thuật toán thay đổi
Kiểm soát thứ tự	Linh hoạt runtime

8 THỰC TẾ & BÀI TẬP

- **Ứng dụng:** ASP.NET MVC Lifecycle, Java HttpServlet, .NET Stream class.
- **BÀI TẬP THỰC HÀNH**
 - 🎯 Bài 1: Refactor Workflow - Xây dựng hệ thống OrderProcess, vẽ UML Before/After.
 - 🎯 Bài 2: Payment Workflow - CreditCard, EWallet, BankTransfer với hook method.
 - 🎯 Bài 3: Advanced - Thiết kế OrderProcessor kết hợp Logging và Retry logic.

10 THẢO LUẬN & KẾT LUẬN

THẢO LUẬN: Template Method Pattern tạo ra rigid system? Khi nào dùng Strategy? Kết hợp Command?

KẾT LUẬN

-  Chuẩn hóa quy trình, Giảm duplicate code, Kiểm soát flow chặt chẽ.
-  Dễ mở rộng, Tuân thủ DRY.